

Laboratory Exercise 09

Topics

1. Using conditional constructs to make decisions (Review Lab03)
2. Using loops for repeated execution of instructions (Review Lab04)
3. Definition and processing of lists and tables (Review Lab08)

Discussion

- A. How do you use conditional constructs to make complex decisions?
- B. A complex decision can be based on more general and more specific conditions. How should the conditional construct handle them?
- C. In the context of loops, what is an **off-by-one error**?
- D. What is the search algorithm used to find an element in an unordered set of elements?
- E. Which construct is best suited for iterating through the values of a table with rows and columns?

Exercises (in red = suggested ones to complete during the lab)

Part 1 – Summary exercises

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least one exercise during the lab session, and the remaining ones at home.

[SUGGESTED] 09.1.1 List of functions. Write a function for each of the following operations on lists of integers, and create a program to test each function. All operations must modify the list passed as a parameter. [P6.4]

- I. Swap the first and last items on the list with each other.
- II. Slide all the items in the list by one position, moving the last item to the first position. For example, the list **1 4 9 16 25** it should become **25 1 4 9 16**.
- III. Replace all even values with 0.
- IV. Replace each element, except the first and last, with the larger of the two elements adjacent to it.
- V. Delete the middle element of the list if it has an odd size, otherwise delete the two middle elements.
- VI. Move all even items to the top of the list, leaving odd items at the bottom, while preserving relative ordering between items.
- VII. Return the second largest value in the list.
- VIII. Return **True** if and only if the list is sorted in ascending order.
- IX. Return **True** if and only if the list contains two duplicate adjacent items.
- X. Return **True** if and only if the list contains duplicate (not necessarily adjacent) items.

09.1.2 Hidden Rules. Given an empty list **l = []**, write a program that fills it with each of the following sequences of values, after understanding the sequence generation rule (if it exists). [R6.1]

- I. **1 2 3 4 5 6 7 8 9 10**
- II. **0 2 4 6 8 10 12 14 16 18 20**
- III. **1 4 9 16 25 36 49 64 81 100**

```

IV.  0 0 0 0 0 0 0 0 0 0
V.   1 4 9 16 9 7 4 9 11
VI.  0 1 0 1 0 1 0 1 0 1
VII. 0 1 2 3 4 0 1 2 3 4

```

09.1.3 Bar chart. Write a program that receives a sequence of real values as input and displays a bar graph that represents them, using asterisks (*) to draw the bars. The output should follow the following format:

```

*****
*****
*****
*****
*****

```

Assume that all values in the input sequence are positive. As a first step, identify the maximum value. The bar that represents this value must be composed of 40 asterisks. The shorter bars must use a proportional number of asterisks to represent the remaining values. [P6.24]

09.1.4 What about negative numbers? Extend the program in Exercise 09.1.3 so that it also works correctly with sequences that may contain negative numbers. [P6.25]

09.1.5 Captions. Extend the program of exercise 09.1.3 by adding a caption next to each bar. The program must request both captions and sequence values from the user as input. The output should follow the following format:

```

Egypt *****
France *****
Japan *****
Uruguay *****
Switzerland *****

```

[P6.26]

09.1.6 List of integers. Write a program that reads from the user a list of positive integers provided on a single line and separated by the ':' character, for example: 3:12:21:8:4:7. The program must print, maintaining the same format:

- I. the numbers entered, excluding the minimum and maximum (e.g., 12:8:4:7);
- II. among the numbers entered, only even numbers (for example, 12:8:4);
- III. Among the numbers entered, only 2-digit numbers (for example, 12:21).

Part 2 – Applications

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least one exercise during the lab session, and the remaining ones at home.

[SUGGESTED] 09.2.1 At the theatre. The map of theater seats is represented by a table with ticket prices for each seat, like this one.

```
10, 10, 10, 10, 10, 10, 10, 10, 10, 10
10, 10, 10, 10, 10, 10, 10, 10, 10, 10
10, 10, 10, 10, 10, 10, 10, 10, 10, 10
10, 10, 20, 20, 20, 20, 20, 20, 10, 10
10, 10, 20, 20, 20, 20, 20, 20, 10, 10
10, 10, 20, 20, 20, 20, 20, 20, 10, 10
20, 20, 30, 30, 40, 40, 30, 30, 20, 20
20, 30, 30, 40, 50, 50, 40, 30, 30, 20
30, 40, 50, 50, 50, 50, 50, 50, 40, 30
```

Write a program that asks the user to choose either:

- a place (providing row and column)
- a price
- or exit from the program.

When the user specifies a place, make sure that it is free and that the coordinates are valid. On the other hand, when the user specifies a price, assign any seat from those available at that price (if any). Mark the seats already sold with a price equal to 0. [P6.27]

09.2.2 Concatenated words. On long train journeys, to pass the time, you can play the game of "concatenated words". The first player says an initial word, then in turn each player must say a word not yet said during the game, whose initial syllable is equal to the final syllable of the previous word. An example of a sequence (in Italian) is: 'gatto', 'torino', 'notte', 'tela', 'lana'. For simplicity, let's assume that all syllables are exactly 2 characters long. So, for example, for 'figli' the final syllable will be 'li' and not 'gli'. Write a program to play one or more sessions of this game. Each session ends when one of the players enters a word already said during the same game, when he enters an incorrectly concatenated word, or when he fails to enter a new word. To leave the game, the player must enter an asterisk (*).

09.2.3 The best customer. A supermarket wants to reward its best customer of the day by displaying their name on a screen inside the store. For this purpose, the names of all customers of the day are stored in one list (`customers`) and, in another list (`sales`), the amount of the shopping made by each of them. Write the `name_of_best_customer(sales, customers)` function that returns the **name** of the customer who spent the highest amount. Then, write a program that asks the cashier to type in the amounts spent by all customers and their names, adding them, after each acquisition, to two separate lists, and then invoke the designed function and display the result. Use the amount 0 as a sentinel. [P6.33]

09.2.4 Spiral. Write a program that asks the user to enter a value N , creates a table of $N \times N$ elements, fills it with a sequence of integers with values between 1 and N^2 , following a spiral order. Finally, the program should display the table thus filled. For example, if $N = 4$, the program should print the table shown on the left, following the directions shown in the table on the right (which should not be displayed by the program).

1	2	3	4
12	13	14	5
11	16	15	6
10	9	8	7

⊙	→	→	↘
↗	→	↘	↓
↑	⊗	↙	↓
↑	←	←	↙

09.2.5 Pet Shop. A pet store wants to give a discount to its customers if they buy one or more animals and at least five other products. The discount is equivalent to 20% of the cost of other products, excluding animals. Write a `discount(prices, is_pet)` function that receives information about a single sale and calculates the value of the discount to apply. For the i -th product, `prices[i]` is the price before discounting, and `is_pet[i]` is a Boolean variable that takes value `True` if the i -th product is an animal. Write a program that requires the cashier to input the price for each product sold, and then, 'Y' if the product is an animal, and 'N' if it is not. Use the price `-1` as a sentinel value. Save the inputs in a list, call the implemented function, and display the discount. [P6.32]

09.2.6 Flood [seen in class, slightly simplified]. Suppose you have a `heights` table of real values that shows the elevation of a terrain at different points in a square area. Write a $N \times N$ function `flood_map(heights, water_level)` that, by processing a `heights` table of $N \times N$ real numbers representing altitudes, and given the `water_level` value of the water level, constructs and returns the map of the corresponding flood; this map is represented by a table of $N \times N$ strings containing an asterisk '*' to indicate that the a certain location is flooded, or blank space ' ' to indicate that the location is dry.

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Then, write a program that builds a 10×10 matrix of heights, reading 100 terrain elevation values in inputs, and visualizes, recalling the previous function, how the ground is flooded when the water

level rises. To do this, divide the range of heights (from minimum to maximum) into 10 intervals of equal size, and for each of these 10 intermediate values (`water_level`) print the map of the corresponding flood. [P6.35]